XORIANT

CLIENT EVALUATION INTERVIEW PLAYBOOK

Collaboration • Manual Testing • Agile Delivery • Defect Handling • Practical Scenarios

Prepared for: Swapnil Bodade
Profile: Test Lead / SDET • 13 years of experience

Plain-English answers grounded in your CV

What this guide is designed to demonstrate
You are not only an automation engineer. You are an experienced Test Lead who understands requirements, asks useful questions, works with developers and Product Owners, makes risk-based testing decisions, communicates defects clearly, and helps the team release with confidence.

Important: Use these as model answers, not memorised scripts. Keep examples truthful to your own experience. Where a process detail is not explicitly stated in your CV, the guide gives a practical approach you can adapt to the client's actual workflow.

How to use this guide

The client feedback points to one main expectation: be an active participant. The interviewer wants to hear how you think, clarify uncertainty, collaborate, and take ownership—not only which tools you know.

Your answer pattern

1. Context: what was happening?
2. Action: what did you personally do?
3. Collaboration: who did you involve and what did you ask?
4. Result: what improved or what decision was made?
5. Learning: what would you carry forward?

Contents

1. Tell me about yourself and your current project

2. Collaboration with developers, Product Owners, analysts and QA

3. Active participation in Agile ceremonies

4. Manual testing process from requirement to release

5. Test design, prioritisation and risk-based testing

6. Defect reporting, triage, retesting and disagreement

7. Realistic client scenarios and practical answers

8. Using automation experience in a manual-testing project

9. Leadership, ownership and communication

10. Questions to ask the client

11. Short answers and phrases to sound engaged

12. Final interview checklist

1. Tell me about yourself and your current project

1. Tell me about yourself.

Interview-ready answer: I have around 13 years of experience in software quality engineering, covering manual testing, test automation, API testing, mobile testing, and test leadership. In my current role at Xoriant, I work on a US Health Insurance Exchange platform supporting state-specific eligibility and enrolment workflows. My responsibilities include UI and API regression automation, framework improvements, Jenkins integration, and collaboration with developers and product stakeholders. Earlier, at TCS, I worked on a wealth-management platform, led a five-member testing team, and participated in test planning, execution, defect management, and coordination with functional and technical teams. I focus on understanding business risk, asking questions early, and helping the team deliver reliable releases.

In simple words: Start with your experience, explain the current product in one sentence, then show how you work with people and contribute to quality.

Practical example: *Do not spend the whole introduction listing tools. Mention Playwright, Selenium, REST Assured or Jenkins only when they support a point.*

2. Explain your current HIX project to a non-technical person.

Interview-ready answer: It is a US health-insurance marketplace where people can compare plans and complete eligibility and enrolment activities. Because rules can differ by state, we need to verify both common journeys and state-specific behaviour. My role is to help ensure important user journeys work correctly through regression testing and automation, while coordinating with development and product stakeholders.

In simple words: Explain the user outcome first; avoid unexplained abbreviations.

3. What was your role as Test Lead, beyond writing scripts?

Interview-ready answer: My role includes understanding testing scope, identifying risk, improving the automation framework, supporting regression execution, reviewing quality signals, helping other engineers, and coordinating with developers and product stakeholders. I try to raise questions early rather than wait until testing is blocked. For a release, I want the team to understand what was tested, what failed, what remains risky, and what decision is needed.

4. Your CV focuses on automation. Can you contribute to manual testing?

Interview-ready answer: Yes. Automation is one part of quality engineering, not a replacement for test analysis or exploratory testing. My experience includes requirement analysis, test planning, functional and regression testing, defect reporting, retesting, and release validation. At Hurix and GlobalStep, I also performed manual testing and worked through defect life cycles. I can apply the same structured thinking to manual testing: understand the requirement, identify scenarios, execute carefully, record evidence, report issues, and communicate risk.

2. Collaboration: developers, PO, analysts and QA

5. Who did you collaborate with in your previous experience?

Interview-ready answer: I collaborated with developers to clarify implementation behaviour and investigate defects; Product Owners or product stakeholders to clarify business rules and acceptance criteria; functional or business analysts to understand requirements; and QA engineers to review scenarios, share findings, and coordinate coverage. In my TCS role, my CV specifically mentions coordinating with functional and technical teams to define testing scope. In my current Xoriant role, I collaborate with developers and product stakeholders in Agile/Scrum ceremonies.

In simple words: Show that collaboration is purposeful: each person helps answer a different question.

6. How do you collaborate with a developer when a requirement is unclear?

Interview-ready answer: First, I check the story, acceptance criteria, existing behaviour, and related test cases. I then explain exactly what is unclear and ask a focused question. For example: 'When the applicant changes the state after entering household details, should the previous state-specific data be cleared or retained?' I confirm the expected behaviour with the Product Owner or analyst and the technical implications with the developer. I document the agreed decision in the story or test notes so the whole team has the same understanding.

In simple words: Do not just say 'I ask the developer'. Explain the question, the right owner, and how you record the answer.

7. How do you work with a Product Owner (PO)?

Interview-ready answer: I involve the PO when I need business intent clarified—for example, eligibility rules, mandatory fields, error messages, or what should happen in an unusual user journey. I explain the risk in business language and, where useful, give examples of possible outcomes. I ask the PO to confirm the expected result, then update the test scenarios and share the decision with the team.

In simple words: PO = helps clarify what the product should do and why.

8. What if the developer says the behaviour is expected, but you think it is a defect?

Interview-ready answer: I avoid making it personal. I bring the requirement, acceptance criteria, steps to reproduce, test data, actual result, expected result, and evidence. I ask the developer to explain the intended behaviour. If the requirement is ambiguous, I ask the PO or analyst to decide. We then classify it as a defect, requirement change, or expected behaviour and document the decision.

In simple words: Focus on evidence and user impact, not on proving someone wrong.

9. How do you make sure you are not sitting silently in meetings?

Interview-ready answer: I prepare by reviewing the story and identifying questions, risks, dependencies, and test ideas. During the discussion I contribute when I can add value: I clarify acceptance criteria, point out a missing negative scenario, raise a dependency, or confirm how a fix will be tested. If I do not understand something, I ask a concise question rather than make assumptions. After the meeting I capture decisions and follow up on open points.

In simple words: Active participation does not mean speaking constantly. It means contributing useful information at the right time.

10. Give an example of a useful question you would ask in refinement.

Interview-ready answer: For a health-insurance enrolment story, I might ask: 'What should happen if eligibility service returns no result or times out?' 'Are the rules identical for all supported states?' 'Which fields are mandatory?' 'What message should the user see?' 'Is the change limited to this journey, or could it affect existing applicants?' These questions help define test coverage before development is complete.

11. How do you collaborate when a defect crosses UI, API and database layers?

Interview-ready answer: I first reproduce the issue and identify the layer where the behaviour diverges. I capture the UI steps, request/response details where permitted, correlation or transaction identifiers, timestamps, and relevant database evidence if access is available. I share a concise summary with the developer and work with the relevant API, UI, or data owner. I avoid assuming the UI is at fault just because the issue is visible there.

12. How do you communicate with a busy developer?

Interview-ready answer: I keep the message short and actionable: impact, environment, reproducible steps, expected versus actual result, evidence, and urgency. I ask for a specific next step or clarification instead of sending a vague message such as ‘it is not working’. If the issue blocks testing, I state the blocker and when I need an update.

3. Active participation in Agile ceremonies

13. What do you contribute in sprint planning?

Interview-ready answer: I review the stories, acceptance criteria, dependencies, and known risks. I ask whether the story is testable, whether test data or environments are ready, and whether API or integration changes need coverage. I estimate testing effort, including negative scenarios, retesting, and regression impact. If a dependency is unclear, I raise it before the team commits.

14. What do you contribute in backlog refinement?

Interview-ready answer: I help make the story testable. I ask questions about business rules, edge cases, permissions, validations, state-specific differences, error handling, and dependencies. I suggest acceptance criteria in Given/When/Then form when that makes behaviour clearer. I also identify test data and environment needs early.

15. What do you say in daily stand-up?

Interview-ready answer: I keep it focused: what I completed, what I will do next, and what is blocked. For example: ‘Yesterday I completed the happy-path and validation checks for the enrolment story. Today I am testing the API error paths and retesting the latest fix. I am blocked because the test environment returns an inconsistent eligibility response; I have shared evidence with the developer and need confirmation on the service status.’

16. How do you contribute in sprint review or demo?

Interview-ready answer: I share the tested scope and important outcomes, not just a pass percentage. I mention the key user journeys covered, defects fixed, remaining limitations, and any risks that need attention. If a result depends on a business rule, I ask the PO to confirm the expected behaviour during the review.

17. What do you discuss in a retrospective?

Interview-ready answer: I focus on improving the process, not blaming individuals. For example, if late requirement changes caused rework, I suggest earlier refinement or adding examples to acceptance criteria. If test data repeatedly blocked execution, I suggest a named owner and preparation checklist. I follow up on agreed actions in the next sprint.

18. What if the team does not invite QA to refinement?

Interview-ready answer: I would explain the impact respectfully: QA involvement early can identify ambiguity and reduce rework later. I would ask to attend refinement for stories with significant testing risk and offer practical contributions, such as edge cases, data dependencies, and acceptance-criteria gaps. I would demonstrate value rather than frame it as a complaint.

4. Manual testing process: requirement to release

A simple end-to-end flow

1. Understand	2. Plan	3. Design
Read story, requirements and acceptance criteria. Ask questions.	Scope, risk, environment, data, dependencies and time.	Scenarios, test cases, expected results and traceability.
4. Prepare	**5. Execute**	**6. Report**
Check build, access, environment and test data.	Run tests; record actual results and evidence.	Log defects; retest fixes; run impact-based regression.
7. Close	**8. Improve**	**Always**
Summarise coverage, pass/fail, blockers and residual risk.	Review escaped defects and improve future coverage.	Communicate early; never hide a blocker or risk.

19. Explain your manual testing process from start to finish.

Interview-ready answer: First I understand the requirement and acceptance criteria, and clarify gaps with the PO or analyst. Next I identify scope, impacted areas, risks, dependencies, test data, and environment needs. I design positive, negative, boundary, and integration scenarios and map them to the requirements. Once the build is available, I perform smoke checks and execute the planned tests, recording actual results and evidence. I log defects with clear reproduction steps, retest fixes, and run regression around impacted areas. At the end, I communicate coverage, open defects, blockers, and residual risks so the team can make an informed release decision.

In simple words: In plain English: understand what should happen, decide what to check, prepare, test, report problems, check fixes, and explain what is still risky.

20. What do you do when a new build arrives?

Interview-ready answer: I verify the build/version and environment, check whether required services and test data are available, and run a small smoke test on critical journeys. If smoke testing fails, I report the blocker before spending time on the full suite. If it passes, I execute the planned story tests and relevant regression.

21. What is the difference between smoke, sanity, retesting and regression?

Interview-ready answer: Smoke testing checks whether the build is stable enough for deeper testing. Sanity testing is a focused check of a particular change or area. Retesting verifies that a specific reported defect has been fixed. Regression testing checks that existing behaviour has not broken because of the change.

In simple words: Example: after a login fix, retesting checks the original login failure; regression also checks logout, session behaviour, and other impacted flows.

22. How do you decide what to test when time is limited?

Interview-ready answer: I prioritise by business impact, likelihood of failure, user frequency, regulatory or financial impact, and dependency risk. I cover critical end-to-end journeys first, then high-risk validations and negative paths, followed by lower-risk areas. I tell the team what is not covered and why, instead of implying that all testing is complete.

23. What if requirements are incomplete but the deadline is close?

Interview-ready answer: I identify the exact gaps and the risks they create, ask the PO or analyst for decisions, and document assumptions only when the team agrees. I can continue testing confirmed behaviour while keeping ambiguous areas visible. I do not silently invent expected results. If a decision cannot be obtained, I communicate the impact and ask the relevant owner to decide how to proceed.

24. How do you test a story when no formal test cases exist?

Interview-ready answer: I use the story, acceptance criteria, design or API contract, and discussions with the team to build a concise checklist. I cover the main flow, invalid inputs, boundaries, permissions, failure paths, and relevant integration points. I record what I tested and the outcome so the work remains traceable and repeatable.

5. Test design, coverage and risk

25. How do you write a good test case?

Interview-ready answer: A good test case has a clear purpose, preconditions, test data, steps, expected result, and traceability to a requirement. It should be understandable by another tester and repeatable. I avoid combining too many unrelated checks in one case and include enough evidence to explain a failure.

26. How do you test a mandatory field?

Interview-ready answer: I check the valid value, empty value, spaces only, minimum and maximum length, invalid format, boundary values, and whether the error message is clear. I also check keyboard behaviour and whether the form submits only when required fields are valid. The exact cases depend on the field's business rules.

27. How do you test a health-insurance enrolment workflow?

Interview-ready answer: I would cover the end-to-end journey: applicant details, household information, state-specific eligibility, plan selection, review, and submission or confirmation—based on the actual product scope. I would include valid and invalid data, required fields, back/next navigation, duplicate submission, service failure, session timeout, and state-specific rules. I would confirm expected outcomes with the PO rather than assume all states behave identically.

28. What is risk-based testing?

Interview-ready answer: It means spending the most testing effort on the areas where failure would be most damaging or likely. In a health-insurance marketplace, eligibility, enrolment, plan selection, and data integrity may be high risk. A low-impact text alignment issue may receive lower priority unless it affects accessibility or blocks a user.

29. How do you ensure requirement coverage?

Interview-ready answer: I map each requirement or acceptance criterion to one or more test scenarios. I track which scenarios passed, failed, were blocked, or were not run. When requirements change, I review the mapping and identify affected tests. This helps show both what is covered and what remains untested.

30. How do you test state-specific behaviour?

Interview-ready answer: I first confirm which rules are common and which differ by state. I use a state-by-scenario matrix to make the differences visible, then test representative common journeys and the specific variations. I confirm expected behaviour with product stakeholders and ensure regression covers states affected by the change.

31. How do you test negative scenarios?

Interview-ready answer: I consider what can go wrong: missing or invalid data, unauthorised access, duplicate actions, unavailable services, timeouts, invalid responses, and interrupted user journeys. I verify not only that the system rejects the request, but also that the message is understandable, data is not corrupted, and the user can recover where appropriate.

6. Defect handling and difficult conversations

32. What details should a good defect contain?

Interview-ready answer: Title, environment/build, preconditions, test data, clear steps, expected result, actual result, severity/priority recommendation, screenshots or logs, and any request/response or transaction identifiers that help investigation. I keep the title specific, such as ‘Eligibility page shows success after service timeout’ rather than ‘Page not working’.

33. What is the difference between severity and priority?

Interview-ready answer: Severity describes how badly the defect affects the system or user. Priority describes how urgently the team should fix it. A defect can be high severity but scheduled later if it affects a rare non-critical path; a visually small issue may have high priority if it blocks a major release or a critical customer journey. The team should agree based on impact and deadlines.

34. A developer cannot reproduce your bug. What do you do?

Interview-ready answer: I recheck the steps, environment, build, account permissions, test data, and timing. I attach evidence and logs, and try to reproduce it together with the developer. If it is intermittent, I record frequency and conditions rather than claim it happens every time. I keep the discussion collaborative and update the defect with new findings.

35. What if a fix passes retesting but breaks another area?

Interview-ready answer: I log or reopen the new issue with evidence, identify the shared component or dependency, and expand regression around the affected area. I discuss the impact with the developer and PO, and update the release risk. A fix is not considered safe only because the original test passes; impacted behaviour also matters.

36. How do you handle a disagreement about defect priority?

Interview-ready answer: I describe the user impact, affected workflow, frequency, workaround, and release consequences. I share evidence and ask the PO or agreed triage owner to make the priority decision. I do not make the discussion personal. Once a decision is made, I record it and ensure any accepted risk is visible.

37. What if the release date is fixed and critical defects remain?

Interview-ready answer: I communicate the facts: which critical journeys are affected, what was tested, the defect impact, available workarounds, and what remains uncertain. I offer options where possible, such as fixing before release, disabling the affected feature, or delaying the release. The authorised business and engineering owners make the release decision; QA provides a clear risk assessment and does not hide the defect.

38. What does defect triage mean?

Interview-ready answer: It is a discussion where the team reviews defects, confirms validity and impact, assigns severity and priority, identifies an owner, and agrees the next action. QA brings reproduction evidence and user impact; development explains technical scope; product stakeholders help determine business urgency.

7. Practical client scenarios

39. A story is marked ‘Ready for QA’ but the acceptance criteria are unclear. What will you do?

Interview-ready answer: I will not begin by guessing. I will list the unclear points, check any related documentation or existing behaviour, and ask the PO or analyst focused questions. I can prepare tests for confirmed behaviour while keeping the unresolved cases visible. I will document the agreed decisions and update the test scenarios.

40. A developer says, ‘QA should figure it out.’ How do you respond?

Interview-ready answer: I would say, ‘I can investigate and propose scenarios, but I want to confirm the intended business behaviour so we do not test against an assumption. I have identified two possible outcomes—could we confirm which one is expected?’ This shows ownership while keeping the right decision with the business owner.

41. The API is down and manual testing is blocked. What do you do?

Interview-ready answer: I confirm whether the service is unavailable for everyone or only the test environment, capture the time and error, and notify the service owner and team. I continue with independent testing where possible, such as UI checks that do not depend on the service or test design. If a stub or mock is approved and available, I use it for limited validation, clearly marking what has not been verified against the real integration.

42. You have ten test cases left and only one hour. How do you proceed?

Interview-ready answer: I rank them by critical business risk and dependency. I execute the highest-risk cases first, reuse reliable data where appropriate, and avoid rushing through checks without evidence. I tell the team which cases were completed and which remain untested, with the reason. I do not report the whole scope as passed.

43. A bug appears only sometimes. How do you investigate?

Interview-ready answer: I record exact steps, timestamps, environment, user/data conditions, frequency, and any relevant logs. I look for patterns such as timing, parallel activity, stale data, session expiry, or service instability. I try controlled repetitions and collaborate with the developer. I label it intermittent if that is what the evidence shows.

44. A production defect escaped testing. How do you respond?

Interview-ready answer: First I help assess impact and support reproduction or containment. Then I identify why the test did not catch it: missing scenario, misunderstood requirement, environment difference, test-data gap, or a regression gap. I add or improve coverage and share the learning without blaming an individual. The aim is to reduce the chance of recurrence.

45. The PO changes a requirement late in the sprint. What do you do?

Interview-ready answer: I clarify the change and its business reason, assess which test cases and areas are affected, and estimate the additional testing and regression effort. I communicate the impact to the team and ask whether scope or timing needs adjustment. I update the test notes and do not assume the old acceptance criteria still apply.

46. You find a defect shortly before a demo. Do you raise it?

Interview-ready answer: Yes, I raise it promptly with clear impact and evidence. I also explain whether a workaround exists and which part of the demo or user journey is affected. The team can then make an informed decision. Hiding a defect until after the demo removes the chance to manage the risk.

47. A test passes in your environment but fails in another environment. What do you check?

Interview-ready answer: I compare build versions, configuration, test data, permissions, service endpoints, feature flags, browser/device, and dependency availability. I capture the differences and work with the environment or development owner. I avoid declaring it an application bug or environment issue until evidence supports the conclusion.

48. How would you test if a third-party eligibility service is unavailable?

Interview-ready answer: I verify the expected fallback behaviour with the PO or service contract. I check timeout handling, user messaging, retries if defined, duplicate submissions, data consistency, logging, and recovery after the service returns. I also confirm that the application does not show a false success or lose user-entered information.

8. Using automation experience in a manual-testing project

49. This project currently has manual testing. Will you be comfortable?

Interview-ready answer: Yes. My priority would be to understand the product, business rules, current test process, and release risks. I can contribute immediately through test analysis, manual execution, defect investigation, and collaboration. My automation background also helps me recognise repeatable regression candidates, but I would not push automation before understanding the team's needs or the client's priorities.

50. Would you try to automate everything?

Interview-ready answer: No. I would first stabilise and understand the manual test coverage. Automation is most valuable for repeatable, high-risk regression, data-driven checks, and stable workflows. Exploratory testing, rapidly changing areas, and subjective usability checks may remain manual. I would discuss cost, maintenance, execution frequency, and team skills before proposing automation.

51. How can automation experience improve manual testing?

Interview-ready answer: It helps me think in terms of repeatability, clear preconditions, test data, expected results, and evidence. It also encourages me to look for API or data-level checks that can isolate the cause of a UI problem. But I still use exploratory thinking and human judgement when requirements are unclear or behaviour is unexpected.

52. Tell me about your Selenium-to-Playwright migration.

Interview-ready answer: In my current Xoriant role, I contributed to migrating from Selenium to Playwright to improve maintainability, stability, and execution efficiency. I helped develop reusable components and work with JavaScript/TypeScript, Cucumber, and API automation. I would explain the migration through the problem it addressed, the approach taken, how compatibility and regression were validated, and the outcome. I would quote specific speed or stability metrics only if I can substantiate them.

53. How do you decide whether a test should be manual or automated?

Interview-ready answer: I consider how often it runs, business risk, stability of the workflow, repeatability, data combinations, execution time, maintenance cost, and whether human judgement is needed. A critical flow run in every regression cycle is a strong candidate; a one-off exploratory check may be better manual.

54. What would you do in your first two weeks on a manual-heavy client project?

Interview-ready answer: I would learn the product and users, review requirements and test assets, understand the release cadence and defect process, meet the PO/analyst/developers/QA, and observe the current execution workflow. I would identify critical journeys, recurring blockers, environment and data dependencies, and reporting expectations. I would first contribute within the existing process, then suggest improvements based on evidence.

9. Leadership, ownership and communication

55. How do you show ownership?

Interview-ready answer: I take responsibility for moving a quality question forward: clarify the requirement, identify the right people, investigate the issue, keep the status visible, and follow through until there is a decision. Ownership does not mean solving everything alone; it means making sure the right next action happens and risks are communicated.

56. How do you mentor a junior tester?

Interview-ready answer: I explain the business purpose first, ask them to propose scenarios, review their reasoning, and help them identify missing positive, negative, and boundary cases. I give specific feedback with examples and encourage them to ask questions. I also explain how to report a defect clearly and how to communicate uncertainty without guessing.

57. How do you handle conflicting priorities from two stakeholders?

Interview-ready answer: I make both requests visible, understand deadlines and business impact, estimate the work, and explain the trade-off. I ask the relevant lead or PO to confirm the order if the priorities conflict. I avoid silently accepting both and then missing a commitment.

58. How do you communicate test status to non-technical stakeholders?

Interview-ready answer: I use plain language: what business journeys were checked, what passed or failed, what is blocked, which defects matter most, and what risk remains. I avoid reporting only tool logs or raw counts. I distinguish ‘not tested’ from ‘failed’ and ‘blocked’ from ‘passed’.

59. Tell me about a time you improved quality.

Interview-ready answer: My CV describes my contribution to Selenium-to-Playwright migration and reusable automation components, as well as CI/CD integration and framework standards in earlier roles. I would choose one real example and explain the previous problem, the action I personally took, how I collaborated with the team, and the evidence of improvement. If I do not have a measured metric, I describe the observable result without inventing a percentage.

60. What if you do not know the answer to a question?

Interview-ready answer: I would be transparent: ‘I have not encountered that exact case, so I would not want to guess. I would first clarify the expected behaviour, check the relevant documentation or logs, and collaborate with the right owner. Based on the evidence, I would propose a test approach and validate it.’

10. Smart questions to ask the client

Ask two or three relevant questions when invited. These demonstrate curiosity and help you understand how to contribute.

• What are the most critical user journeys or business risks the QA team is responsible for?

• How are requirements and acceptance criteria currently shared with QA?

• At what stage does QA usually join refinement and planning discussions?

• What is the current defect triage process, and who makes the final priority decision?

• How do you measure testing completeness and release readiness?

• What are the main challenges in the current manual testing process—test data, environment stability, changing requirements, or regression time?

• How do QA, developers, and the Product Owner typically collaborate when expected behaviour is unclear?

• Are there any state-specific, integration, or third-party workflows that need particular attention?

• What would success look like for the person joining this team in the first 60–90 days?

Strong closing question
‘Based on our discussion, is there any area of my experience you would like me to explain in more detail, especially around collaboration, manual testing, or defect handling?’

11. Phrases that show active collaboration

Clarifying a requirement: “I want to confirm the expected business behaviour before I finalise the test cases.”

Raising a risk: “One risk I see is that this change may affect the existing enrolment flow. Can we confirm the regression scope?”

Asking for evidence: “Could you help me understand the expected result? I can share the exact steps and current behaviour.”

Disagreeing respectfully: “I see it differently based on the acceptance criteria. Can we review the evidence together?”

Being proactive: “I noticed this dependency may block testing, so I am raising it now rather than waiting until execution.”

Reporting a blocker: “I am blocked by the test environment. I have captured the error and contacted the owner; meanwhile I can continue with these independent checks.”

Showing ownership: “I will follow up with the relevant owner and update the story once the expected behaviour is confirmed.”

Acknowledging uncertainty: “I do not want to assume the rule. I have two possible interpretations—could the PO confirm which one is correct?”

Giving status: “The critical path has passed, two negative cases remain, and one integration case is blocked. I will share the risk before sign-off.”

Avoid these weak responses

Weak response	Stronger response
“I just test what is assigned.”	“I review the story, identify risks, and clarify gaps before execution.”
“Developer told me it is not a bug.”	“We reviewed the evidence and asked the PO to confirm the expected behaviour.”
“Everything passed.”	“The planned critical tests passed; these cases remain blocked or untested.”
“I don't know.”	“I have not seen this exact case. I would investigate using the requirement, evidence and the right stakeholders.”
“Automation is better than manual.”	“I choose the approach based on risk, repeatability, stability and business need.”

12. Final interview checklist

■ Prepare a 60–90 second introduction without listing every tool.

■ Explain the HIX product and why state-specific regression matters.

■ Prepare one real collaboration story involving a developer or product stakeholder.

■ Prepare one real defect story: evidence, discussion, decision, and result.

■ Explain manual testing from requirement clarification to release summary.

■ Know smoke, sanity, retesting, regression, severity, priority, and risk-based testing.

■ Prepare an example of a useful question you asked or a risk you raised early.

■ Be clear about your personal contribution versus what the whole team did.

■ Do not invent metrics, tools, responsibilities, or project details not in your experience.

■ If a question is unclear, ask the interviewer to clarify before answering.

■ Use examples: ‘In my Xoriant HIX project…’, ‘At TCS…’, ‘In my manual testing work at Hurix/GlobalStep…’ only when factually accurate.

■ End answers with the outcome, decision, or learning—not just the activity.

The main message to leave with the client
“I am a hands-on Test Lead who combines business understanding, structured testing, automation knowledge, and active collaboration. I ask questions early, use evidence to resolve disagreements, make risks visible, and work with the team to protect critical user journeys.”

CV grounding note

This playbook is tailored to the uploaded five-page CV: Xoriant / GetInsured-Vimo HIX (multi-state health insurance, UI/API regression, Selenium-to-Playwright migration, Jenkins and Agile collaboration); Synechron / Asurion (automation services, reusable frameworks, CI/CD, code reviews and mentoring); TCS / Morgan Stanley Coverage Central (five-member team, test planning, defect logs, API testing and coordination); Microsoft Office mobile testing; Hurix / Kitaboo (manual execution, test cases, defect lifecycle and Appium); and GlobalStep / BigFish (functional, system, regression, integration and defect reporting). Questions that extend beyond the CV are framed as practical approaches to adapt to the client’s actual process—not as claims that you have already done each exact activity.